

Design Note — Lexical vs. Semantic Retrieval for News Recommendation

CS4.406 Information Retrieval & Extraction — Assignment 1

1 What We Built

News recommendation differs structurally from catalog domains like retail or streaming: item relevance half-life is hours, not months, so item cold-start is the *default* state, not an edge case (Wu et al. 2020; Kruse et al. 2024) — motivating content-based candidate retrieval, lexical (BM25) and semantic (embeddings), over collaborative filtering. This project implements both methods, plus one shared evaluation harness, over MIND and EB-NeRD under a unified schema and temporal-split protocol (ADR-001, ADR-002), comparing methods and datasets on identical footing rather than via method-specific pipelines. Both were pushed through to real Codabench leaderboard submissions on both competitions (§3.5). Each dataset’s weakest link was then chased with a follow-on candidate — NRMS-lite for MIND (§3.6), a LightGBM ranker for EB-NeRD (§3.7) — both of which became this project’s real leaderboard wins.

2 Choices We Made

Data & splits. Official train/validation splits are used as-is (7-day windows) — the only feasible option given data budgets (ADR-001). A unified three-table schema (articles, impressions, user_history) with mandatory core + dataset-specific optional fields lets one retrieval and evaluation code path run against both datasets (ADR-002).

Lexical retrieval. BM25Okapi over unweighted concatenated title+abstract query text (full click history, stopwords removed), scored as a precomputed sparse weight matrix rather than `rank_bm25`’s naive `get_scores()` — the latter measured at ~10hr projected vs. ~80s actual at MINDsmall-dev scale, a real infeasibility, not a style preference (ADR-005, ADR-006).

Semantic retrieval. One embedding model computed over both datasets (rejecting EB-NeRD’s provided embeddings + a separate MIND model, to keep the single-code-path comparison valid) — `paraphrase-multilingual-MiniLM-L12-v2`, chosen over `multilingual-e5-small` after a same-category vs. different-category cosine-similarity discrimination check: MiniLM’s gap (0.069, 2.4x ratio) is real and usable; e5-small’s `passage:-`prefixed convention compresses nearly all pairs into a narrow 0.75–0.78 band (gap 0.017, ratio 1.02x) — e5’s asymmetric query/passage objective doesn’t fit this project’s symmetric user-profile-vs-article use case (ADR-008). ANN backend is brute-force cosine similarity — 0.99ms/query at MINDsmall scale, 2.44ms/query at MINDlarge’s 255,990 users — FAISS stays unjustified at this scale (ADR-008 + Addendum; revisited at 10x scale, §5). User representation is mean-pooled, L2-renormalized embeddings of full click history; zero-history users get an explicit all-zero vector, mirroring BM25’s own cold-start reporting posture.

Rejecting EB-NeRD’s provided embeddings (above) was argued, not measured, until a follow-up (ADR-008 Addendum) found the provided artifact wins `recall@K/accuracy` CI-clear (AUC 0.5453 vs. 0.5430) but loses `Diversity@10` — a real, EB-NeRD-only trade-off, not a blanket verdict.

Evaluation. Two distinct questions are measured, not conflated: *recall@K* (does retrieval surface the clicked article out of the *whole* corpus, $K \in \{50, 100, 200\}$) and *Q4’s ranking metrics* — AUC, MRR, nDCG@5, nDCG@10, intra-list diversity, novelty, catalog coverage — over candidates *already listed in each impression*. Ties break via a deterministic per-impression-seeded pseudo-random permutation (cold users produce exact ties); $K = 10$ for diversity/novelty. Every metric reports 95% bootstrap CIs ($n = 2000$), sliced warm/cold (history length ≥ 5 = warm), except coverage, a point estimate only — set-union statistics are mechanically biased under with-replacement resampling (ADR-007).

3 Observations

3.1 Recall@K — candidate generation over the whole corpus

Dataset (split)	Method	recall@50	recall@100	recall@200
MIND-small dev, overall	BM25	0.73%	1.50%	2.62%
MIND-small dev, overall	Embed	0.90%	1.60%	2.78%
MIND-small dev, warm	BM25	0.73%	1.54%	2.73%
MIND-small dev, warm	Embed	0.93%	1.67%	2.92%
MIND-small dev, cold	BM25	0.72%	1.19%	1.78%
MIND-small dev, cold	Embed	0.65%	1.10%	1.81%
EB-NeRD-demo val (warm only*)	BM25	1.01%	2.13%	4.02%
EB-NeRD-demo val (warm only*)	Embed	0.32%	0.94%	2.57%
EB-NeRD-small val (warm only*)	BM25	0.72%	1.44%	2.77%
EB-NeRD-small val	Embed	0.14%	0.43%	1.21%

*EB-NeRD’s active-user-filtered bundles have zero cold-start users (min history length = 5) by construction. Source: `experiments/{bm25,embed}_{mind,ebnerd,ebnerd_small}_2026-08-10/`. Not run at MINDlarge scale — flagged in §6, not approximated.

3.2 Q4 ranking metrics — AUC / MRR / nDCG (bootstrap 95% CI)

Dataset	Method	AUC overall (CI)	AUC warm	AUC cold
MIND-small dev	BM25	0.569 (0.567–0.571)	0.577	0.524
MIND-small dev	Embed	0.634 (0.632–0.636)	0.644	0.574
MINDlarge dev	BM25	0.570 (0.569–0.571)	0.578	0.522
MINDlarge dev	Embed	0.634 (0.633–0.634)	0.643	0.575
EB-NeRD-demo val	BM25	0.533 (0.528–0.537)	— (=overall)	n/a
EB-NeRD-demo val	Embed	0.544 (0.539–0.549)	—	n/a
EB-NeRD-small val	BM25	0.529 (0.527–0.530)	—	n/a
EB-NeRD-small val	Embed	0.543 (0.541–0.544)	—	n/a

On MIND (both scales), embeddings’ AUC CI does not overlap BM25’s on either cohort, and widens the relative warm/cold disparity rather than closing it (embeddings lift warm AUC ~ 0.065 – 0.067 over BM25 but cold by only ~ 0.050 – 0.053). On EB-NeRD, embeddings’ AUC CI also doesn’t overlap BM25’s — a modest (~ 0.011 – 0.014) edge — despite BM25 winning recall@K by 1.6–2.3x on the *same* corpus (§3.1): recall@K asks whether the click is anywhere in the whole catalog, AUC asks how well the method ranks EB-NeRD’s already-curated, shorter candidate lists (median 9–12 vs. MIND’s 23) — see §6. MRR/nDCG@5/@10 follow AUC’s direction on both datasets (e.g. MINDlarge nDCG@10: 0.390 vs. 0.350; EB-NeRD-small: 0.459 vs. 0.454).

3.3 Diversity / novelty / coverage (K=10, point estimate for coverage)

Dataset	Method	Diversity@10	Novelty@10	Coverage@10
MIND-small dev	BM25	0.837	16.30	0.083
MIND-small dev	Embed	0.827	16.15	0.078
MINDlarge dev	BM25	0.836	18.09	0.067
MINDlarge dev	Embed	0.828	17.76	0.065
EB-NeRD-small val	BM25	0.795	17.17	0.206
EB-NeRD-small val	Embed	0.789	17.19	0.205

BM25 and embeddings sit within ~ 1 point of each other on every beyond-accuracy metric, on every corpus — accuracy differs meaningfully, beyond-accuracy behavior barely does (n up to 376,471 impressions, so not automatically noise, but not the large effect AUC shows).

3.4 System cost

BM25 at MINDlarge scale: 1.70s index build, 23.5MB sparse weight matrix, ~ 9.8 min projected full retrieval for 255,990 users (ADR-006 Addendum). Embeddings: 264.4s to encode+cache all 72,023 articles, ~ 10.4 min projected retrieval (ADR-008 Addendum). Both stay local on an 8GB machine at this scale — only EB-NeRD’s blind test split (13.5M impressions, no local ground truth) required Kaggle.

3.5 Leaderboard results (Q5)

Competition	Method	Score	Submission ID	Date
MIND (competitions/13967)	Embed (MiniLM)	0.6195	886468	2026-08-12 13:01
MIND (competitions/13967)	Cohort-gated combiner	0.6192	896696	2026-08-22
MIND (competitions/13967)	NRMS-lite (Cand. J)	0.6462	901961	2026-08-26
EB-NeRD (competitions/2469)	Embed (MiniLM)	0.5404	888045	2026-08-13 23:08
EB-NeRD (competitions/2469)	Contrastive vector	0.5404	896072	2026-08-21 13:01
EB-NeRD (competitions/2469)	GBDT ranker (Cand. K, §3.7)	0.7542	907863	2026-08-29 23:01

Both used the embedding method — the clear winner on local dev-set AUC for both datasets (§3.2). EB-NeRD’s first score (0.5404) is close to local validation AUC on `ebnerd_small` (0.5430) — a coherence check, not a claim the two are defined identically.

Second MIND submission — cohort-gated combiner, essentially flat. Local validation predicted a CI-clear win at every stage ($+0.0027$ MINDsmall-dev, $+0.0019$ MINDlarge-dev re-verification, ADR-010’s Addendum), but the real result was **0.6192 vs. 886468’s 0.6195 (–0.0003)** — flat, not the predicted win. No pipeline defect found (line count, malformed-permutation, all 32 N89741 cases clean); most likely a genuine population difference, since MIND’s dev/test splits are disjoint calendar weeks (ADR-001) and this edge was built around MINDlarge-dev-specific properties.

Second EB-NeRD submission — contrastive vector, real test set. Both submissions round to the same Score (0.5404) — checked, not assumed identical: different checksums, 99.48% of impressions rank differently for the same ID, confirming independent runs. Codabench’s per-submission detail view (grouped by date, “50% of the testset”) resolves the tie: contrastive leads 5/8 days and the mean AUC (0.5402 vs. 0.5397, $+0.0005$, sign flips three times day-to-day) — real but far smaller than the ~ 0.0023 **CI-clear gap** ADR-008’s Addendum measured locally (0.5453 vs. 0.5430).

Third MIND submission — NRMS-lite (Candidate J), a real win. A trainable title/history self-attention architecture (Wu et al. 2019, GloVe-init, ADR-012), unlike every frozen-embedding candidate below. Trail: MINDsmall-dev 0.6391 $\rightarrow$ MINDlarge-dev 0.6579 (margin *widened*, unique among all candidates) $\rightarrow$ real leaderboard **0.6462**

(+0.0267 over the original submission — compressed from the dev screen, but a real win, unlike the combiner above). A narrow-impact scoring-catalog bug (initially estimated 32/2,370,727 impressions, later measured directly at 2,087/2,370,727, 0.088%) was found and fixed post-submission; a corrected resubmission scored an identical **0.6462**, confirming the bug never put the result in question (ADR-012).

3.6 MIND candidate search: ten approaches, and what worked

Ten approaches screened on MINDsmall-dev vs. baseline (0.634, §3.2) before J, all atop *frozen* embeddings with a shallow head. Only G’s routing won CI-clear overall (didn’t hold at real scale), only F’s combiner won CI-clear on the cold cohort. Evidence: ADR-010 (D–H)/011 (I)/012 (J).

Cand.	Method	Result
A	Entity embeddings (TransE)	0.553, loss
B	50/50 BM25+embedding hybrid	0.626, loss
C	Recency-weighted embedding query	0.627, loss
D	Symbolic category/entity overlap	0.613, loss
E	Train-popularity only	0.532, loss
F	7-feature LR combiner	0.626 loss; 0.593 cold win
G	Cohort-gated routing	0.637 local win; 0.6192 real (flat)
H1	LogisticRegression, balanced	0.632, loss
H2	HistGradientBoosting	0.599, loss
I	Attention re-ranker, 5 epochs	0.623, loss
I-long	Same, 30 epochs	0.621, worse than I
I-pop	I-long + raw popularity feature	0.513, inconclusive
J	Trainable title+history encoders	0.658 local win; 0.6462 real win

3.7 EB-NeRD candidate search: Candidate K (GBDT learning-to-rank)

EB-NeRD’s deployed score (0.5404) sits *below* the challenge’s own popularity baseline (0.5970): in-view candidates are drawn from the same front page at the same moment (median age 3.7h, 58% under 6h), leaving content similarity little to discriminate on. Unlike MIND’s ten-way search (§3.6), EB-NeRD’s search is a single candidate, K — a LightGBM learning-to-rank model over 65 engineered behavioural/temporal features — chosen because the RecSys Challenge 2024 organizers’ own report states most real competitors used GBDT ensembles, not neural architectures; a better-embeddings option and an NRMS-style option (as Candidate J did for MIND) were both rejected on that evidence before building K (ADR-013).

A real per-impression bug was found and fixed mid-session. Short-term (recency-decayed) features were computed once per *user* against a shared reference time, not once per *impression* — effectively static. Fixed (`compute_short_term_features`, referenced to each impression’s own timestamp); a second, independently found leak-safety gap in the same path (no upper bound excluding a future-dated history click) was closed at the same time. Local `ebnerd_small` AUC moved from **0.7514–0.7528 to 0.7581–0.7597**, CI-clear (ADR-013’s 2026-08-27 Addendum).

Trained at real `ebnerd_large` scale on Ada (IIIT-H’s SLURM cluster): 12,063,890 train / 12,566,385 validation impressions, the latter scored in full via a streaming path. Result: local validation AUC **0.7590 (95% CI 0.7588–0.7593)** — essentially unmoved by ~10.7x more training data, a real null result on scale for this feature set (ADR-013’s 2026-08-29 Addendum).

Real Codabench result: 0.7542 (§3.5) — +0.2138 over the previous deployed EB-NeRD submission, +0.1572 over the challenge’s popularity baseline. Compression from local `ebnerd_large` validation was small (0.7590 → 0.7542, –0.0048) — the first EB-NeRD/MIND candidate here whose CI-clear local win transferred largely intact, rather than evaporating (MIND’s cohort-gated combiner) or thinning to near-nothing (EB-NeRD’s contrastive vector), plausibly because its edge is structural, leak-safe item-level signal rather than a validation-population-specific pattern (ADR-013’s 2026-08-30 Addendum).

Feature importance at real scale: freshness (led by `article_age_h`) is **33.3%** of gain; long-term interest 14.1%; **short-term interest only 0.8%** (down from an already-small 1.9% pre-fix at small scale). The BlackPearl-derived long/short-term hypothesis this candidate was built to test is therefore **falsified**: what wins is freshness *relative to other candidates in the same list* and `context_embed_sim` (what the user is reading right now), not modelled user interest. Withholding position features *improved* AUC by a CI-clear +0.0011 — the shipped arm (`K_rank_nopos`) omits them and needs no such caveat.

4 Anti-Gaming and Leakage (Q9)

Serving-time-unavailable features — measured, not just avoided. Novelty’s popularity signal comes from the **train split only**, Laplace-smoothed, never the evaluation split (ADR-007). EB-NeRD’s raw `articles.parquet` also carries three lifetime aggregate fields — `total_inviews`, `total_pageviews`, `total_read_time` — unavailable at serving time; `src/datasets/ebnerd.py` never reads them (grep-verified). `scripts/run_leakage_ablation.py` measures what including them would buy: an untuned 50/50 blend of the deployed embedding score with a min-max-normalized leak signal, on `ebnerd_small` validation (ADR-009):

Metric	Without leaky features	With leaky features (50/50 blend)	CIs overlap?
AUC	0.5430 (0.5415–0.5445)	0.5662 (0.5646–0.5678)	No
MRR	0.3437 (0.3421–0.3452)	0.3638 (0.3623–0.3653)	No
nDCG@5	0.3804 (0.3784–0.3823)	0.4034 (0.4016–0.4051)	No
nDCG@10	0.4591 (0.4574–0.4607)	0.4783 (0.4767–0.4799)	No

Every metric moves by a CI-clear margin from exposure to these fields alone (AUC +0.023 absolute, $\sim 4.3\%$), untuned. $\sim 48\%$ of articles were median-imputed, not zero-filled, so this gap is plausibly a conservative underestimate — direct, measured evidence for excluding these fields entirely (ADR-009).

Leakage-boundary test. `tests/integration/test_leakage.py` asserts no user’s history click occurs at or after their own earliest impression time. **Exists and passes for EB-NeRD** (demo train + validation splits). **Explicitly skip-marked for MIND:** MIND provides only a static per-user article-ID list, no per-click timestamps, so the temporal boundary can’t be independently verified from the data. An acknowledged gap, not an oversight — MIND’s temporal safety instead rests on the dataset’s own documented split construction (ADR-001).

5 Where It Breaks at 10×

Every real scaling failure so far was invisible at the previously-exercised scale and appeared at a 5–14x jump, not a 2x one — a pattern, not a one-off.

MINDsmall → **MINDlarge** ($\sim 5x$ users: 50,000 → 255,990; $\sim 14x$ impression-candidate rows) hung in a per-token Python loop never exercised at MINDsmall’s counts; the first fix projected to **$\sim 27GB$** (measured, not estimated) from one Python object per repeated ID string, resolved via categorical dtype (12.5x reduction, $\sim 2.2GB$); a second bottleneck (`series.map()` on categorical data, >1hr, zero progress) was found only by profiling the stuck process. **MINDlarge_test (2.37M)** → **ebnerd_testset (13.5M, 5.7x further)** hit a third, structurally identical failure: a converter materializing a full `list[dict]` before writing projected to $\sim 16GB$, invisible at the smaller scale — fixed by a generator consuming one row at a time. **ebnerd_small** → **real ebnerd_large training on Ada** ($\sim 52x$ further train impressions) hit a fourth, real SLURM-confirmed OOM: an un-split 65-feature train matrix ($\sim 11.5GB$) was never freed, and a column-sliced copy needed by one arm coexisted with it ($\sim 22.7GB$ from four arrays alone). Fixed by freeing the matrix once training finished, capping the training sample $4M \rightarrow 2.5M$ impressions, and raising `--mem-per-cpu` $4G \rightarrow 8G$ (four further non-modelling Ada incidents the same session in ADR-013’s 2026-08-29 Addendum).

Projecting one more 10x from the largest verified scale is inference, not measurement: (1) retrieval time scales roughly linearly with users, so 10x users alone stays locally feasible (~ 100 min); (2) brute-force cosine is $O(\text{users} \times \text{articles})$, so a 10x corpus alongside 10x users is 100x compute (~ 10.4 min $\rightarrow \sim 17$ hours), forcing a revisit of ADR-008’s “FAISS unjustified” call; (3) every real bug so far shared one shape — a per-row-object or full-materialization pattern past an unexercised scale — and nothing guarantees another path is safe at a further 10x.

6 Limitations and Open Questions

- **EB-NeRD’s real result (0.7542, §3.7) did not reach this project’s internally-discussed 0.80 target — stated plainly.** The target referenced a leakage-dependent figure: the RecSys Challenge 2024 winner scored 88.64 in the organizers’ own ablation arm, but **76.99** once those same features were found to leak future information and removed (an 11.65-point drop; arXiv:2409.20483). 76.99 is the honest ceiling this project can defensibly compare against; 0.7542 sits **0.0157 below it**, on features that passed the same leakage audit every candidate here uses.
- **True (zero-history) cold-start is a shared ceiling**, not solved by either method (MIND cold-cohort recall@200: BM25 1.78% vs. embed 1.81%); EB-NeRD’s cold cohort is structurally empty in both `demo/small` (min history length = 5, `large` unverified), and recall@K (§3.1) was never run at MINDlarge scale.
- **The Q9 leakage-boundary test is real but partial** (passes EB-NeRD, skip-marked MIND, no per-click timestamps; §4); its ablation magnitude is a lower bound, not a maximum, from an untuned 50/50 blend. Two other choices rest on a proxy, not a gold standard: the encoder’s discrimination check (ADR-008) and the stopword list (ADR-005).
- **The EB-NeRD recall-vs-AUC disagreement isn’t fully explained** (§3.2): BM25 wins whole-corpus recall 1.6–2.3x, yet embeddings edge the curated candidate-list AUC — plausibly a diffuse mean-pooled query, not isolated by experiment.
- **Local wins compress at real scale, but don’t always vanish:** MIND’s combiner and EB-NeRD’s contrastive vector compressed to flat; Candidate J widened at MINDlarge-dev, then compressed at the real leaderboard but stayed substantial (+0.0267 over baseline, §3.5/3.6); **Candidate K compressed least of all** (0.7590 local $\rightarrow$ 0.7542 real, -0.0048 , §3.7) — architecture/signal type plausibly affects compression (inference, $N = 4$). Relatedly, the BlackPearl long/short-term hypothesis (A1, ADR-013) is **falsified**: short-term interest fell to 0.8% of Candidate K’s gain *after* the recency fix made it more correctly computed, not less important.
- **A numpy/CPython 3.14 mismatch produced provably wrong array results (ADR-013)**, fixed by rebuilding on Python 3.11 — unclear if any earlier number predates the fix.